FinMarket-Enterprise

Phase II — Database and Security Integration

Member 2: @DerenDaniel-Humber — Lead Management (CRM)

Branch: `feature/leads-p2-m2`
Assigned module: `backend/src/modules/leads/`
Primary ownership: Lead database persistence, lead lifecycle management, consent tracking, advisor ownership enforcement, and CRM security.

---

1. Project Overview

FinMarket-Enterprise is a financial advisory marketing and client acquisition platform designed to help financial advisors manage prospective clients, organize marketing campaigns, deliver authorized marketing communications, automate follow-up activities, and monitor marketing performance.

The application consists of five main modules:

1. Identity and Advisor Management
2. Lead Management (CRM)
3. Marketing Campaigns and Templates
4. Email and Facebook Delivery
5. Automation and Analytics

During Phase I, the Lead Management module was implemented using temporary in-memory storage. Core API endpoints were established for creating, retrieving, updating, searching, and filtering leads. Initial Mongoose models and business validation rules were also introduced.

Phase II focuses on replacing temporary lead storage with MongoDB, implementing persistent data relationships, protecting lead information, enforcing advisor ownership, and maintaining compatibility with the other application modules.

The existing Lead Management architecture and API contracts should be preserved wherever practical.

1.1 Phase II Objectives

The following responsibilities must be completed:

- Replace in-memory lead storage with MongoDB using Mongoose.
- Implement persistent Lead records with appropriate schemas and indexes.
- Maintain relationships between Leads and AdvisorProfiles.
- Protect lead endpoints using shared JWT authentication middleware.
- Enforce role-based authorization and advisor ownership.
- Prevent unauthorized access to private lead information.
- Maintain lead lifecycle validation.
- Enforce duplicate contact prevention.
- Implement secure marketing consent and unsubscribe tracking.
- Provide reliable lead filtering, searching, and pagination.
- Preserve existing Phase I API functionality.
- Prepare lead data for use by Campaigns, Delivery, and Automation.
- Implement Postman and automated tests covering database persistence, validation, and authorization.

The Lead Management module must be independently developable and testable without requiring completion of the other feature modules.

---

2. Development Environment and Team Independence

The Dockerized backend and MongoDB infrastructure will be prepared by Member 1 and merged into `main` before Phase II feature development begins.

Once the infrastructure is available, the repository must be cloned or updated from the latest `main` branch.

Development must occur in:

`feature/leads-p2-m2`

The shared Docker configuration, database connection, JWT authentication middleware, role constants, and error-handling contracts must be reused.

Member 2 must not independently modify or replace shared infrastructure.

The Lead Management module must remain responsible for its own:

- Mongoose models.
- Database repositories.
- Services and business rules.
- Controllers and routes.
- Module-specific validation.
- Authorization integration.
- Unit and integration tests.
- Module documentation.

Changes to shared files such as `backend/src/app.js`, `backend/src/routes/index.js`, `backend/src/config/`, or shared authentication middleware should be avoided unless explicitly coordinated with Member 1.

2.1 Independent Development Requirements

The module must be developed without waiting for other members to finish their feature implementations.

The following approach must be followed:

1. Existing Lead routes and service contracts must be retained wherever practical.
2. The database-backed repository must replace the temporary in-memory repository without changing unrelated modules.
3. Stable identifiers must be used to reference AdvisorProfiles.
4. Test fixtures may be used to create sample Advisor and User records directly in the test database.
5. Authentication and authorization tests may use mock authenticated request contexts or locally generated test JWTs, consistent with the shared security contract.
6. Campaign, Delivery, and Automation modules must not be required to run for core Lead Management tests.
7. The module must expose stable service and repository interfaces that other modules can use after integration.

The completed module must run with the shared MongoDB connection and authentication middleware.

---

3. Shared User Roles and Permissions

The application supports three roles:

3.1 Administrator — `ADMIN`

Administrators manage platform-level user accounts, access permissions, and administrative operations.

Lead Management access must follow explicitly authorized administrative permissions.

The Administrator role must not be treated as an unrestricted bypass unless such access has been intentionally defined in the shared authorization policy.

3.2 Financial Advisor — `ADVISOR`

Financial Advisors own their prospective client records.

An authenticated Advisor must be allowed to:

- Create leads under their own AdvisorProfile.
- View their own leads.
- Retrieve details of their own leads.
- Update their own lead records.
- Change lead statuses according to lifecycle rules.
- Record and update marketing consent.
- Search and filter their own leads.

An Advisor must not be permitted to access another Advisor's private lead records simply by supplying a different `advisorId`.

3.3 Marketing Specialist — `MARKETING_SPECIALIST`

Marketing Specialists are responsible for permitted marketing activities on behalf of assigned Advisors.

However, an accepted advisor-specialist relationship does not automatically grant access to private CRM records.

For Phase II, the following policy must be applied:

- Marketing Specialists must not create or modify leads by default.
- Marketing Specialists must not retrieve unrestricted lead directories.
- Marketing Specialists must not access private lead contact information solely because they have accepted an advisor invitation.
- Campaign and Delivery modules may obtain eligible lead information through controlled backend service operations.
- Any future direct Lead Management permission for specialists must be explicitly defined and enforced.

Campaign-specific specialist assignments are managed by Member 3.

Advisor-specialist invitation relationships are managed by Member 1.

Member 2 must enforce Lead Management permissions without implementing these other modules' responsibilities.

---

4. Database Identifiers and Relationships

The following identifier meanings must be used consistently.

4.1 Identifier Definitions

Field	Meaning	Referenced Collection
`userId`	Unique identifier of a registered user	`users`
`advisorId`	Unique identifier of an AdvisorProfile	`advisorProfiles`
`leadId`	Unique identifier of a Lead	`leads`
`specialistUserId`	User ID belonging to a Marketing Specialist	`users`

`campaignId`	Unique identifier of a Campaign	`campaigns`
`ownerId`	User ID of the owner where defined	`users`

Important distinction:

`advisorId` refers to `AdvisorProfile._id`, not `User._id`.

For example:

- `User._id = A`
- `AdvisorProfile._id = B`
- `AdvisorProfile.userId = A`
- `Lead.advisorId = B`

Every lead must reference the AdvisorProfile that owns it.

This convention must be preserved in the Mongoose model, service logic, repository queries, and API responses.

4.2 Required Relationships

AdvisorProfile → Lead

Relationship: One-to-many.

One AdvisorProfile may own multiple leads.

Each Lead must belong to exactly one AdvisorProfile.

Required reference:

`Lead.advisorId → AdvisorProfile._id`

Lead → Campaign

Campaigns select eligible leads using criteria such as:

- Advisor ownership.
- Lead interest.
- Lead status.
- Marketing consent.

- Unsubscribe status.

The Lead Management module must make these fields available for authorized Campaign audience selection.

A direct `campaignId` field is not required in the Lead model because one lead may qualify for multiple campaigns.

Campaign audience selection logic belongs to Member 3.

Lead → DeliveryJob

Delivery jobs may reference leads through:

`DeliveryJob.leadId → Lead._id`

The Lead Management module must provide accurate lead existence, ownership, email, and consent information.

Delivery eligibility enforcement and message processing belong to Member 4.

Lead → ActivityEvent

Lead creation and certain lifecycle changes may generate business events for automation.

The agreed event contract must support references such as:

- `eventType`
- `entityType`
- `entityId`
- `ownerId`
- `metadata`

For a lead event:

- `entityType = "LEAD"`
- `entityId = Lead._id`
- `ownerId = User._id` of the owning Advisor

The owning Advisor's User ID must be resolved through the AdvisorProfile relationship.

Event processing, automation execution, and ActivityEvent persistence belong to Member 5.

Member 2 must not independently implement the Automation module.

---

5. Member 2 — Assigned Phase II Responsibilities

5.1 Lead Mongoose Model and MongoDB Persistence

Owned directory: `backend/src/modules/leads/`

The existing Lead model must be reviewed and updated for MongoDB persistence.

Collection:

`leads`

Recommended fields include:

- `_id`
- `advisorId`
- `name`
- `email`
- `phone`
- `interest`
- `status`
- `marketingConsent`
- `consentRecordedAt`
- `unsubscribedAt`
- `source`
- `notes`
- `createdAt`
- `updatedAt`

The existing Phase I schema fields and accepted enum values must be preserved unless a change is necessary and documented.

The implementation must include:

1. Required field validation.
2. Correct Mongoose field types.
3. Appropriate default values.
4. Allowed enum values.
5. Timestamp management.
6. AdvisorProfile references.
7. Useful indexes.
8. Duplicate contact protection.

The `advisorId` field must use a Mongoose ObjectId reference targeting AdvisorProfile.

The model must not assume that a valid ObjectId automatically means the referenced AdvisorProfile exists.

Reference existence must be verified through the appropriate shared Identity lookup or an agreed relationship validation contract.

5.2 Replace the In-Memory Repository

Primary file: `backend/src/modules/leads/leadRepository.js`

The existing repository currently stores Lead records in an in-memory Map.

This temporary implementation must be replaced with Mongoose database operations.

Required repository functionality includes:

- Create a lead.
- Find a lead by ID.
- Find leads by advisor.
- Update an existing lead.
- Search and filter leads.
- Count matching leads.
- Retrieve paginated results.
- Check for duplicate contacts.

Existing service and repository interfaces should be preserved where practical.

All database operations must use Mongoose queries.

The following practices must be followed:

- Database queries must remain in the repository layer.
- Controllers must not directly query MongoDB.
- Services must enforce business rules.
- Mongoose validation must be applied consistently.
- Repository operations must handle asynchronous database access.
- Records must remain available after server restart.

The current `leadRepository.js` implementation must no longer rely on a JavaScript Map for production data storage.

5.3 Lead Creation and Advisor Ownership

Existing endpoint:

`POST /api/v1/leads`

Lead creation must require JWT authentication.

Only an authorized Advisor may create a lead under their own AdvisorProfile, unless an explicit administrative permission is defined.

The creation workflow must:

1. Authenticate the requesting user.
2. Verify the `ADVISOR` role or an explicitly authorized administrative permission.
3. Resolve the AdvisorProfile associated with the authenticated user.
4. Determine the correct `advisorId`.
5. Validate the submitted lead data.
6. Check for duplicate contacts.
7. Apply default lifecycle and consent values.
8. Save the validated record to MongoDB.
9. Return the created lead.

The authenticated AdvisorProfile ID must be used as the ownership reference.

If `advisorId` is included in a request body for compatibility, it must match the authenticated advisor's profile or be rejected.

A client must not be able to create leads under another advisor by changing an identifier in the request body.

5.4 Lead Retrieval and Privacy

Existing endpoints:

- `GET /api/v1/leads`
- `GET /api/v1/leads/:id`

Lead retrieval must require authentication and authorization.

The implementation must verify that the requested Lead belongs to the authenticated Advisor.

The following restrictions must be enforced:

- Advisors may retrieve only their own lead records.
- Specialists may not retrieve private Lead records by default.
- Unauthenticated requests must be rejected.
- Unauthorized access to another advisor's Lead must be rejected.
- Invalid identifiers must produce structured validation errors.
- Missing records must produce appropriate not-found responses.

Lead contact information must not be exposed through unrestricted public endpoints.

Sensitive CRM data must be returned only when the caller has the required permissions.

Authorized internal services may retrieve limited lead information needed for campaign targeting or delivery eligibility.

Such access must use explicit internal contracts rather than bypassing validation and authorization rules.

5.5 Lead Updates and Lifecycle Enforcement

Existing endpoint:

`PATCH /api/v1/leads/:id`

The existing lead lifecycle must be preserved:

`NEW → CONTACTED → CONSULTATION_REQUESTED → CONVERTED`

A Lead may also have additional supported statuses if they are explicitly defined in the existing Phase I model. Existing valid enum values must not be silently removed.

The update workflow must:

1. Authenticate the requesting user.
2. Retrieve the existing Lead.
3. Verify ownership or explicitly authorized access.
4. Validate the submitted changes.
5. Reject unsupported fields.
6. Prevent unauthorized advisor reassignment.
7. Validate any status changes.
8. Save the updated record to MongoDB.
9. Update the modification timestamp.

The `advisorId` must not be changed through ordinary Lead updates.

A lead assigned to one advisor must not be reassigned to another advisor unless a separate authorized reassignment workflow is explicitly introduced.

Lifecycle transition rules

The valid lifecycle transitions must be documented and enforced.

Recommended transitions:

Current Status	Allowed Next Status
NEW	CONTACTED
CONTACTED	CONSULTATION_REQUESTED
CONSULTATION_REQUESTED	CONVERTED
CONVERTED	No further progression

Changes that skip required lifecycle stages should be rejected unless a documented business exception is implemented.

These transition restrictions are a Phase II enhancement; the existing Phase I implementation primarily validates allowed status values.

The existing Lead status values must be preserved when introducing transition validation.

5.6 Duplicate Lead Prevention

The existing duplicate-contact rule must be preserved and enforced reliably through MongoDB.

A duplicate should be identified using:

AdvisorProfile ID + normalized email address

For example:

Advisor A may have a lead with the email:

`client@example.com`

Advisor B may independently have a lead with the same email.

However, Advisor A must not be able to create a second Lead with the same normalized email.

Recommended database protection:

- A compound unique index using `advisorId` and normalized email.
- Case-insensitive normalization before persistence.
- Duplicate-key error handling with an appropriate HTTP 409 response.

Email normalization must be consistent for both creation and updates.

Duplicate prevention must not depend only on reading every Lead into application memory.

5.7 Marketing Consent and Unsubscribe Tracking

Marketing consent is required to protect recipients from unauthorized marketing communications.

The existing consent-related fields must be retained:

- `marketingConsent`
- `consentRecordedAt`
- `unsubscribedAt`

The following rules must be enforced:

1. `marketingConsent` must be a Boolean value.
2. A new lead must not be treated as having consented unless consent is explicitly recorded.
3. The date consent was obtained must be recorded.
4. Withdrawal of consent must be recorded.
5. A lead who has unsubscribed must not be treated as eligible for email marketing.
6. Consent status must remain available for authorized Campaign and Delivery operations.
7. Consent changes must persist in MongoDB.

Re-enabling marketing consent must follow a documented consent-recording policy and must not occur implicitly when unrelated lead information is updated.

The current Phase I consent handling may be retained and adapted for persistent storage where appropriate.

The Leads module is responsible for accurately storing and validating consent.

The Campaigns module is responsible for selecting eligible audiences.

The Delivery module is responsible for verifying consent again before a message is processed.

This separation must be maintained.

5.8 Searching, Filtering, and Pagination

Existing endpoint:

```
GET /api/v1/leads
```

The existing search and filtering functionality must continue working.

Supported filters should include:

- `advisorId`
- `status`
- `marketingConsent`
- `source`
- `interest`
- `search`
- `page`
- `limit`

The implementation must:

- Validate query parameters.
- Enforce advisor ownership regardless of client-supplied filters.
- Perform filtering through MongoDB queries.
- Use database-backed pagination.
- Return stable pagination metadata.
- Prevent excessively large page sizes.
- Apply predictable sorting, such as newest records first.

Recommended response structure:

```
None
{
  "data": [],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 0,
    "totalPages": 0
  }
}
```

Existing response conventions should be preserved wherever possible to maintain compatibility.

Filtering must not load every database record into application memory before pagination.

5.9 Secure Data Handling and Authorization

The Lead Management module must use the shared security middleware implemented by Member 1.

No separate JWT implementation should be created inside the Leads module.

Required enforcement includes:

- Authentication for all Lead Management endpoints.
- Role-based restrictions.
- Advisor ownership checks.
- Validation of ObjectIds.
- Validation of accepted request fields.
- Protection against mass assignment.
- Prevention of unauthorized advisor reassignment.
- Controlled exposure of contact information.
- Safe handling of database errors.
- Appropriate handling of missing records and duplicate conflicts.

The Lead model must not contain unnecessary credential or authentication data.

User passwords, tokens, and other Identity credentials must not be stored inside Lead records.

A Marketing Specialist's accepted invitation must not automatically grant direct CRM access.

The backend must not rely on frontend role checks for security.

5.10 Internal Contracts for Other Modules

The Lead Management module provides information required by Campaigns, Delivery, and Automation.

These integrations must be supported without implementing the other modules' business logic.

A. Campaigns Integration

Member 3 must be able to obtain authorized audience information using lead attributes such as:

- Advisor ownership.
- Interest.
- Status.
- Marketing consent.
- Unsubscribe status.

Member 2 must maintain the Lead schema and an agreed lookup/query interface.

Member 3 remains responsible for campaign-specific audience selection and campaign eligibility rules.

B. Delivery Integration

Member 4 must be able to validate:

- Lead existence.
- Advisor ownership.
- Recipient email.
- Marketing consent.
- Unsubscribe status.

Member 2 must expose or preserve the agreed lead lookup interface.

Member 4 remains responsible for delivery eligibility, queue management, processing, and delivery status tracking.

C. Automation Integration

Member 5 must be able to process business events associated with Lead creation and lifecycle changes.

The shared event contract must be documented.

Recommended event types include:

- `LEAD_CREATED`
- `LEAD_STATUS_CHANGED`
- `CONSULTATION_REQUESTED`

The actual event names must match the event types supported by the Automation module.

Member 2 must identify the business operations that should produce these events and provide the required Lead identifiers and metadata.

Member 5 remains responsible for ActivityEvent persistence, trigger matching, scheduling, duplicate prevention, and automation execution.

During independent development, event delivery may be simulated or handled through a shared interface.

A required Lead operation must not fail simply because an unfinished Automation module is unavailable.

Final cross-module event integration can be completed during integration testing.

---

6. API Compatibility

The following Phase I endpoints must remain available:

Method	Endpoint	Purpose
POST	`/api/v1/leads`	Create Lead
GET	`/api/v1/leads`	List, search, and filter Leads
GET	`/api/v1/leads/:id`	Retrieve Lead
PATCH	`/api/v1/leads/:id`	Update Lead

These endpoints must be updated to use MongoDB and the shared authentication/authorization system.

The existing route paths should remain unchanged.

No additional public API endpoints are required solely to complete the MongoDB migration.

Additional endpoints may be proposed when necessary, but shared route changes must be coordinated with Member 1.

The existing API response formats should be retained wherever practical.

Any necessary breaking changes must be documented.

---

7. Testing Requirements

The Lead Management module must be tested using Postman and appropriate automated tests.

Testing must use a real MongoDB database, with isolated fixtures and a reliable cleanup procedure.

At minimum, the following scenarios must be verified.

7.1 Database Persistence

- Creating a Lead stores it in MongoDB.
- A stored Lead can be retrieved by ID.
- Lead updates are persisted.
- Data remains available after backend restart.
- Invalid records are rejected.
- Database validation rules are enforced.

7.2 Advisor Ownership

- An Advisor can create a Lead for their own AdvisorProfile.
- An Advisor can retrieve their own Lead.
- An Advisor can update their own Lead.
- An Advisor cannot retrieve another Advisor's private Lead.
- An Advisor cannot update another Advisor's Lead.
- An Advisor cannot reassign Lead ownership through an ordinary update.
- A supplied `advisorId` cannot override authenticated ownership.

7.3 Authentication and Roles

- Missing JWT is rejected.
- Invalid JWT is rejected.
- Expired JWT is rejected.
- Marketing Specialists cannot access unrestricted CRM endpoints.
- Unrelated Advisors cannot access each other's Leads.
- Authorized requests succeed.

7.4 Validation and Duplicates

- Missing required fields are rejected.
- Invalid email addresses are rejected.
- Invalid ObjectIds are rejected.
- Unsupported fields are rejected.
- Duplicate email under the same Advisor is rejected.
- The same email may exist under different Advisors if all other rules are satisfied.
- Invalid status values are rejected.
- Invalid lifecycle transitions are rejected.

7.5 Marketing Consent

- Consent defaults are applied correctly.
- Explicit consent is recorded.
- Consent timestamps are stored.
- Consent withdrawal is recorded.
- Unsubscribed leads are identified correctly.
- Consent changes remain persistent.
- Unauthorized users cannot modify consent records.

7.6 Search and Pagination

- Advisor filtering works correctly.
- Status filtering works correctly.
- Interest filtering works correctly.
- Marketing consent filtering works correctly.
- Search matches appropriate Lead fields.
- Pagination returns correct page data.
- Pagination counts are accurate.

- Invalid pagination parameters are rejected.
- Query results do not expose Leads belonging to unrelated Advisors.

7.7 Integration Contract Tests

- Lead records reference valid AdvisorProfiles.
- Campaign audience queries can use the agreed Lead fields.
- Delivery eligibility lookups can retrieve the necessary consent information.
- Lead event metadata follows the shared Automation event contract.
- Core Lead operations remain functional when other feature modules are mocked.

---

8. Documentation and GitHub Requirements

Member 2 must provide module-specific documentation explaining:

- Lead Management purpose and responsibilities.
- MongoDB Lead schema.
- Field definitions and validation rules.
- AdvisorProfile relationship.
- Identifier conventions.
- Supported lifecycle statuses.
- Valid status transitions.
- Duplicate-email prevention.
- Marketing consent and unsubscribe behavior.
- Authentication and authorization requirements.
- API endpoint usage.
- Search, filtering, and pagination.
- Internal integration contracts.
- Postman test scenarios.
- Any necessary changes from Phase I.

The module documentation must clearly state:

`advisorId` references `AdvisorProfile._id`, not `User._id`.

No database credentials, JWT secrets, or sensitive test-user passwords should be committed to GitHub.

Development must occur in:

`feature/leads-p2-m2`

Changes must be committed with meaningful commit messages.

A pull request must be submitted targeting `main`.

The pull request must include:

- A summary of completed functionality.
- Relevant schema and repository changes.
- Authentication and authorization integration.
- Tests performed.
- Any necessary setup instructions.
- Known limitations, if applicable.

The module must remain independently testable before being merged.

---

9. Completion Criteria

Member 2's Phase II responsibilities will be considered complete when:

- The Lead model is integrated with MongoDB using Mongoose.
- In-memory Lead storage has been replaced with persistent database operations.
- Lead records reference valid AdvisorProfiles.
- `advisorId` is consistently interpreted as `AdvisorProfile._id`.
- Required fields, defaults, and validation rules are enforced.
- Duplicate contacts are prevented per advisor.
- Existing Lead creation, retrieval, and update endpoints remain functional.
- Lead endpoints require authentication.
- Advisor ownership is enforced by the backend.
- Marketing Specialists cannot access private CRM records by default.
- Unauthorized and cross-advisor access is rejected.
- Lead status values and lifecycle transitions are validated.
- Marketing consent and unsubscribe tracking are persistent.

- Lead searching and filtering are performed through MongoDB queries.
- Database-backed pagination works correctly.
- Lead information is accessible through agreed internal contracts for other modules.
- Event information needed for Automation is defined.
- No unrelated module's business logic has been implemented or modified unnecessarily.
- Postman and automated tests cover successful and unsuccessful operations.
- Module documentation is complete.
- Changes are committed and submitted through a pull request.

Expected outcome: The Lead Management module will operate as a persistent, secure CRM system using MongoDB and Mongoose. Financial Advisors will be able to manage their own prospective clients, while unauthorized users will be prevented from accessing or modifying private lead information. Campaigns, Delivery, and Automation will be able to use the agreed Lead data contracts without requiring changes to the module's ownership boundaries.